autotest-lab ・ 一般論

GUI自動操作の技術

Web・iOS・Android・Mac・Windows を一望する

主要なライブラリとツールを、対象OS・書く言語・画面の掴み方・AIエージェントとの相性で比べる。

公開情報にもとづく一般論。各ツールの最新状況は公式ドキュメントで確認すること。

全体像

画面の「掴み方」は3通り

A. アプリの内側から

テストがアプリと同じプロセスで動き、部品を直接たどる。
例：Espresso、Flutter integration_test

強み 速い・待ちが安定
弱み アプリの外（通知・設定・他アプリ）は触れない

B. 部品の木（DOM・アクセシビリティ）

OSやブラウザが公開する部品の木から、IDや名前で探して操作する。
例：Selenium、Playwright、Appium、UI Automator、XCUITest、pywinauto、macOSのAX API

強み 主流。テキストで扱えるのでAIとも相性がよい
弱み IDの付け方で安定度が決まる

C. 画像と座標

スクリーンショットを見て、画像認識や座標でクリックする。
例：PyAutoGUI、SikuliX、AIのcomputer use

強み どのOS・どのアプリでも動く
弱み 遅い。解像度やテーマの違いで壊れやすい

「1つのスクリプトで複数OS」を狙う方法：Appium（OSごとのドライバを差し替え）、Maestro（YAMLのフロー）、Robot Framework（キーワードの層でライブラリを包む）、画像ベース。ただし部品の名前やIDはOSごとに違うため、共通化できるのは「手順」までで、「部品の指定」はOS別になりやすい。

主要ツール

比較表 1：Web とモバイル

ツール	対象	書く言語	仕組み	特徴・注意
Selenium	Web（主要ブラウザ）	Java, Python, C#, JS, Ruby	W3C標準のWebDriver。各ブラウザのドライバ経由。WebDriver BiDiへ移行中	最も歴史が長く情報が多い。待ち処理は自分で書く場面が多い
Playwright	Web（Chromium・Firefox・WebKit）	TS/JS, Python, Java, .NET	ブラウザを直接制御するライブラリ＋テストランナー（Microsoft）	自動待機、録画からのコード生成、トレース。ネイティブアプリは対象外
Cypress	Web（主にChromium系・Firefox）	JS/TS	ブラウザの中でテストを実行	開発者体験が良い。複数タブなどに制約
Espresso	Android（自アプリ内）	Kotlin, Java	アプリと同じプロセスで動く（掴み方A）	速く安定。アプリの外は操作できない
UI Automator	Android（端末全体）	Kotlin, Java	アクセシビリティ情報（掴み方B）	通知・設定・他アプリも操作できる
XCUITest	iOS（macOSも）	Swift, Objective-C	Appleの XCTest。アクセシビリティ情報	Xcodeが必要＝Macが必要
Appium	iOS・Android・Mac・Windows ほか	Java, Python, JS, C#, Ruby	WebDriverの口＋OSごとのドライバ（UiAutomator2、XCUITest、Mac2、Windows）	1つのAPIで複数OS。下で動くのは各OSの公式の仕組み
Maestro	iOS・Android・Web	YAML	宣言的なフローを書く	書くのが簡単。複雑な分岐や計算は苦手

主要ツール

比較表 2：デスクトップと、OSを横断するもの

ツール	対象	書く言語	仕組み	特徴・注意
pywinauto	Windows	Python	UI Automation（UIA）または Win32 API	手軽。UIAは遅い部品や取りにくい部品がある
FlaUI	Windows	C#（.NET）	UIA	.NETでの定番
WinAppDriver	Windows	WebDriver対応の各言語	UIAをWebDriverの口で公開（Microsoft）	本家の開発は停滞気味。Appium経由で使われることが多い
macOS AX API	macOS	Python（pyobjc）, Swift	AXUIElement（アクセシビリティ）	このリポジトリの macos/ の実験。権限の許可が必要
Appium Mac2	macOS	Appiumの各言語	XCTestを使うドライバ	Xcodeが必要
AppleScript / JXA	macOS	AppleScript, JavaScript	System Events 経由でアクセシビリティ	手軽だが遅く、壊れやすい
Robot Framework	ライブラリしだい	キーワード（表形式）＋ Python	Selenium・Playwright・Appium などを包む層	読みやすい。中身は包んだライブラリに依存
PyAutoGUI / SikuliX	Windows・Mac・Linux	Python, Java	座標・画像認識（掴み方C）	どこでも動くが壊れやすい
ノーコード／RPA	製品しだい	録画・ブロック	内部は上のいずれかの仕組み	書き手を選ばない。中身が独自形式だとAIや外部ツールと連携しにくい

比較表（追加）

準備の手間と実行の速さ 1：Web とモバイル

ツール	必要なマシン	入れるもの・準備	準備	速さ	速さの理由
Selenium	1台（Windows・Mac・Linux）	言語のライブラリ。ブラウザのドライバは自動で取得（Selenium Manager）	◎	○	1操作ごとにHTTPでやりとりする
Playwright	1台（Windows・Mac・Linux）	pip install と playwright install（ブラウザを取得）。Dockerイメージもある	◎	◎	ブラウザと常時接続。自動待機で無駄な待ちが少ない。並列実行が標準
Cypress	1台	npm install	◎	○〜◎	ブラウザの中で動く
Espresso	1台＋Android端末かエミュレーター	Android Studio・SDK。アプリのソースとビルドが必要	○	◎	アプリと同じプロセスで動き、画面が落ち着くのを自動で待つ
UI Automator	1台＋Android端末かエミュレーター	Android SDK。テスト用アプリのビルドが必要	○	○	アクセシビリティ経由
XCUITest	Mac＋iOS端末かシミュレーター	Xcode。実機ではApple Developerの署名が必要	△	○	アプリと別プロセス。起動は遅め
Appium	テスト実行PC＋Appiumサーバー＋端末（iOSはMacが必須）	Node.js、Appium本体、OSごとのドライバ。iOS実機は署名、AndroidはSDK。バージョンの組み合わせ管理	△	△	HTTP越しに、さらにOSのドライバを経由する二段構え
Maestro	1台＋端末（iOSはMac）	CLI（Java）	○	○	フローを一手ずつ解釈し、自動で待つ

準備：◎ 1台にインストールするだけ ○ 少し手間 △ 複数の部品や署名などが要る ／ 速さ：◎ 速い ○ 普通 △ 遅め × 遅い （一般的な目安。アプリや画面によって変わる）

比較表（追加）

準備の手間と実行の速さ 2：デスクトップほか

ツール	必要なマシン	入れるもの・準備	準備	速さ	速さの理由
pywinauto	Windows 1台	pip install	◎	○	UIAは部品が多い画面で遅くなる
FlaUI	Windows 1台	NuGet で追加	◎	○	UIA（Win32の直接呼び出しは速い）
WinAppDriver	Windows 1台	WinAppDriver本体と開発者モード	○	△〜○	HTTP越しにUIAを使う
macOS AX API	Mac 1台	pyobjc。アクセシビリティの権限を許可	○	○	OSの仕組みを直接呼ぶ
Appium Mac2	Mac 1台	Xcode、Appium、Mac2ドライバ	△	△	HTTP越しにXCTestを経由
AppleScript / JXA	Mac 1台	追加なし（権限の許可のみ）	◎	△	1操作ごとにアプリ間でやりとりする
PyAutoGUI / SikuliX	1台（画面が見えている必要あり）	pip install など。解像度・画面ロックに注意	○	△	画面全体から画像を探す
AIのcomputer use	1台＋AIのAPI	APIキーと実行環境	◎	×	1操作ごとにスクリーンショットをAIが判断（数秒）

準備：◎ 1台にインストールするだけ ○ 少し手間 △ 複数の部品や署名などが要る ／ 速さ：◎ 速い ○ 普通 △ 遅め × 遅い （一般的な目安。アプリや画面によって変わる）

準備を軽くする、一般的な手段

どのツールでも変えられない制約：iOSのアプリを動かすにはMac（Xcode）が要る。それ以外の「台数」や「つなぐ手順」は、仕組みの選び方でかなり減らせる。

1台で完結させる

テストを書く場所と動かす場所を同じPCにする。Web・Windows・Macのアプリは、多くのツールで1台で済む。

コンテナで配る

ツールとブラウザを丸ごとイメージにする（PlaywrightのDockerイメージなど）。版の組み合わせ違いが起きない。

自動で取得させる

ドライバやブラウザを、ツールが版に合わせて自動で取ってくる（Selenium Manager、playwright install）。

クラウドの端末を借りる

実機やブラウザをクラウドで借りる（各社の端末クラウド）。手元のMacや端末の管理が不要になる。

シミュレーターを使う

日々の回帰テストはエミュレーター・シミュレーターで回し、実機は節目だけにする。

準備をスクリプト化する

インストールから接続確認までを1コマンドにし、CIで毎回同じ手順を再現する。

AI

AIエージェントとの相性

ツール・方式	AIがスクリプトを書く	AIが直接操作する	ポイント
Playwright	◎ 情報が多く、生成精度が高い	◎ 公式のPlaywright MCP（部品の木をテキストで渡す）	Web なら現時点でいちばん AI と相性がよい
Selenium	◎ 情報が最も多い	○ コミュニティのMCPなど	古い書き方が混ざりやすい点に注意
Appium	○ コードは書ける。部品のIDは実機の情報が必要	○ MCPサーバーあり（公式・コミュニティ）	画面の木をAIに見せる仕組みとセットで使う
Espresso / XCUITest	○ コードは書ける	△ ビルドが必要で試行に時間がかかる	開発者がアプリと一緒に書く用途に向く
pywinauto / macOS AX	○	○ Windows向けMCPなど／AXの木をテキスト化すれば可能	このリポジトリの ax_dump.py が「木のテキスト化」の例
画像・座標	△	◎ computer use（画面を見てクリック）	どこでも動くが遅く、費用もかかる
非公開の独自形式	× 学習データにない（仕様書を渡せば一部可）	△ APIやMCPを公開すれば可	AIに使わせるには、仕様を文章で渡す・口を開ける工夫が要る

◎ とても向いている ○ 向いている △ 工夫が要る × 難しい （2026年時点の一般的な見立て）

Q&A

Playwright とは？

Pythonで書ける？ → はい、その理解で合っています

Microsoft製のブラウザ自動操作ライブラリ。TypeScript/JS、Python、Java、.NET の4言語に公式対応。

ただし、言語で少し差がある

テストランナー（@playwright/test、並列実行・UIモードなど）はNode.js版の機能。Python版は pytest と組み合わせる（pytest-playwright）。操作のAPIはほぼ同じ。

対象はWeb（ブラウザ）だけ

iOS・Android・Windows・Macのネイティブアプリは操作できない。そこはAppiumや各OSの仕組みの担当。

```
# 準備
pip install playwright pytest-playwright
playwright install      # ブラウザを取得

# 操作を録画してコードを作る
playwright codegen --target python URL

# test_login.py
def test_login(page):
    page.goto("https://example.com")
    page.get_by_label("メール").fill("a@b.c")
    page.get_by_role("button",
                     name="ログイン").click()
    expect(page.get_by_text("ようこそ")) \
        .to_be_visible()

# 実行
pytest
```

操作対象の決め方：「IDマッチング」は手段の1つ

> SeleniumやPlaywrightは「IDマッチング」？ → 正確には「部品の木の属性で探す」方式。IDはその属性の1つで、IDが無くても役割・表示名・テキスト・構造で探せる。Playwrightは「役割＋表示名」とテスト用IDを推奨している。

決め方	例	安定度	注意
テスト用のID	Web: data-testid ／ Android: resource-id ／ iOS: accessibilityIdentifier ／ Windows: AutomationId	◎	開発者が付ける必要がある。画面に出ないので翻訳やデザイン変更に強い
役割＋表示名	get_by_role("button", name="ログイン")	◎〜○	利用者に見えるとおりに指定。アクセシビリティの質に依存
表示テキスト	「ログイン」と書かれた部品	○	言語の切り替えや文言変更で壊れる
構造（木の中の位置）	CSSセレクタ、XPath、「3番目のボタン」	△	レイアウト変更で壊れる。自動生成のidやclassも不安定
画像マッチ＋相対座標	正解画像を探し、その中の決まった点をクリック	△	木が無いアプリでも使える。解像度・テーマ・文言で壊れる
画面の座標だけ	(320, 480) をクリック	×	画面サイズが変わると即壊れる

OS別：部品の木と、IDの呼び名

対象	部品の木（取り出す仕組み）	開発者が付けるID	IDが無いときの代替	木が取れない例
Web	DOM（必ずある）	id、data-testid	役割＋表示名、テキスト、CSS・XPath	canvasで描く画面（ゲーム・一部のグラフ）
Android	アクセシビリティ（UI Automator）	resource-id	content-desc（読み上げ用ラベル）、text、class	ゲームエンジン、独自描画
iOS	アクセシビリティ（XCTest）	accessibilityIdentifier	accessibilityLabel、部品の種類	ゲームエンジン、独自描画
Windows	UI Automation（UIA）	AutomationId	Name、ControlType、ClassName	独自描画、UIA対応が弱い古いUI部品
macOS	アクセシビリティ（AX API）	AXIdentifier	AXRole＋AXTitle／AXDescription	独自描画。AXIdentifierが無いアプリは多い
Flutter・React Native など	各フレームワークがOSの木に変換	Semantics、testID など	表示名	変換の出来しだい

ポイント：標準の部品を使っていれば、IDが無くても「木」はある。困るのは、木そのものが無い（独自描画の）画面。

「Webは簡単、ネイティブはIDが要る」は合っている？

おおむね合っている。ただし「IDが無いと動かない」のではなく「IDが無いと壊れやすい」。本当に動かないのは木が無いとき。

Webが扱いやすい理由

DOMが必ずあり、IDが無くてもCSS・テキスト・役割で特定できる。
ブラウザの開発ツールで、誰でも木をすぐ確認できる。

ただしWebにも落とし穴

フレームワークがビルドのたびに変えるidやclassは不安定。
canvas描画、iframe、Shadow DOM は工夫が要る。→ data-testid や役割で指定する。

ネイティブで起きること

IDが無いと、表示名や木の中の位置に頼る。多言語化やレイアウト変更で壊れやすい。
ゲームなど独自描画の画面には木が無いので、画像に頼るしかない。

打ち手

開発チームにテスト用IDを付けてもらう。読み上げ対応（アクセシビリティ）の改善と同じ作業なので、品質向上として頼みやすい。
付けられない画面だけ画像マッチを使う。

AIが画面の構成をつかむ方法

方法	AIに渡すもの	向き・不向き	例
部品の木をテキストで	DOMやアクセシビリティのスナップショット	正確で安い。IDや役割をそのままスクリプトに使える。木が無いと使えない	Playwright MCP、Appiumのpage source、ax_dump.py
スクリーンショットを見る	画面の画像	どの画面でも使える。座標がずれやすく、遅く、費用が高い	computer use
両方を重ねる	画像に部品の番号を重ねたもの（Set-of-Mark）	木の正確さと、見た目の理解を両立	多くのAIエージェントが採用
画像から部品を取り出す	画像を解析した部品の一覧（文字認識・物体検出）	木が無いアプリ向け。精度は画面しだい	OmniParser など
事前に探索して地図を作る	画面ごとの棚卸し＋画面遷移図	スクリプト作成の材料になる。画面の変更検知にも使える	ax_crawl.py、アプリのクローラー

探索して地図を作る → テストの意図を渡す → AIがスクリプトを書く → 実行する → 失敗時は木を取り直す

壁打ちの論点：AI時代に選ばれやすい自動操作の条件

1. テキストのコードで書ける

AIが読める・直せる・差分をレビューできる。録画やブロックだけの形式は不利。

2. 公開されていて情報が多い

AIの生成精度は、世の中にある例の量でほぼ決まる。

3. 画面をテキストで取り出せる

部品の木（DOM・アクセシビリティ）をテキストで渡せると、AIが部品を正しく特定できる。

4. AIから直接呼べる口がある

MCPなどで「操作」と「画面の取得」をAIに開くと、探索的なテストも任せられる。

問い：AIが「書き手」を代わるなら、ノーコードの価値は下がる。一方で、OSごとの違いを吸収する層や、安定して動かす実行基盤（端末の管理・並列実行・結果の記録）の価値は残るのでは？ ――どこを自前で持ち、どこを公開の仕組みとAIに任せるか。